

Department of Computer Science and Engineering

1. Subject Code: **AIS-401** Course Title: **Computational Intelligence**
2. **Expected Effort:** Total: 150 Hours Pacing: 10 Hours / Week Duration: 15 Weeks
3. **Course Structure**
 5 Units × 30 Hours (lectures, source-code reading, laboratory, PoW)

1. Course Description

This course treats large language models as an engineering substrate rather than a research subject. Students begin at the hardware boundary VRAM, memory bandwidth and kernel-level attention — and work upward through production retrieval, agentic tool use, high-throughput local inference, and finally the operational layer of tracing, evaluation and cost control. Every unit is assessed by a measured artefact: savings, recall, throughput or cost must be demonstrated with numbers, not asserted.

2. Course Outcomes

After successful completion of the course, students will be able to:

- CO1.** Profile and optimise GPU memory bottlenecks using mixed precision.
- CO2.** Architect hybrid retrieval pipelines combining dense, sparse and re-ranking stages.
- CO3.** Deploy autonomous agent workflows using graph-based state machines.
- CO4.** Serve open-weight small language models locally with continuous batching.
- CO5.** Instrument LLM applications with semantic caching, evaluation and tracing.

3. Detailed Syllabus

Unit	Contents	Hrs
1	AI Compute, VRAM Hierarchy & GPU Profiling <i>Module A: Hardware Fundamentals</i> Matrix multiplication as the unit of AI compute. GPU architecture: streaming multiprocessors, warps, memory hierarchy, and the distinction between VRAM-bound and compute-bound workloads. Arithmetic intensity and the roofline model. <i>Module B: PyTorch Memory Management</i> Tensor allocation and the caching allocator, activation memory, gradient and optimiser state accounting, gradient checkpointing, and reading a memory timeline. <i>Module C: Precision and Kernel Efficiency</i> FP32, TF32, FP16 and BF16 numerics. Mixed-precision training with loss scaling. FlashAttention and IO-aware kernel design. Fused optimisers and kernel launch overhead. Hands-on Laboratory & Proof-of-Work (PoW) • PoW: Profile a standard PyTorch training loop, implement mixed precision, and measure exact VRAM savings and speedup on a T4/L4 class GPU. • Produce a roofline-style analysis identifying whether a given model step is memory- or compute-bound. • Apply gradient checkpointing and quantify the memory/compute trade-off.	30
2	Applied RAG & Hybrid Retrieval Systems <i>Module A: Ingestion and Chunking</i> Document parsing, layout-aware extraction, chunking strategies (fixed, recursive, semantic, hierarchical), metadata design, and deduplication. <i>Module B: Vector and Sparse Retrieval</i> Embedding models and dimensionality trade-offs. Vector databases (Qdrant, Milvus): HNSW and IVF indexes, filtering, quantised vectors. Sparse retrieval with BM25 and learned sparse (SPLADE) representations. <i>Module C: Fusion and Re-ranking</i> Reciprocal Rank Fusion, weighted hybrid scoring, cross-encoder re-ranking, query rewriting and expansion. Retrieval evaluation: recall@k, MRR, nDCG. Hands-on Laboratory & Proof-of-Work (PoW) • PoW: Build a hybrid RAG pipeline over 500 PDF pages using Qdrant and a cross-encoder re-ranker, proving higher recall than standard cosine-similarity search. • Construct a labelled retrieval evaluation set and report recall@k and nDCG per configuration. • Tune HNSW parameters and chart the recall versus latency frontier.	30

Unit	Contents (continued)	Hrs
3	AI Agents & Tool Calling (MCP) <i>Module A: Structured Outputs and Function Calling</i> JSON schema-constrained generation, tool/function definitions, argument validation, and failure handling for malformed outputs. <i>Module B: Reasoning Loops and State Machines</i> ReAct (Reasoning + Acting) loops, reflection and self-correction. LangGraph state machines: nodes, edges, conditional routing, checkpointing and human-in-the-loop interrupts. <i>Module C: The Model Context Protocol</i> MCP architecture: servers, tools, resources and prompts. Exposing local capability safely, transport options, permissioning, and multi-agent hand-off patterns. Hands-on Laboratory & Proof-of-Work (PoW) • PoW: Create a dual-agent system (Researcher + Summariser) using LangGraph and MCP that autonomously queries a local SQLite database and drafts a report. • Author a custom MCP server exposing a filesystem or API capability with scoped permissions. • Instrument the agent loop to log every tool call, argument and outcome for replay.	30
4	Edge AI, SLMs & High-Throughput Inference <i>Module A: Small Language Models</i> Open-weight model families and capability/size trade-offs. Prompt-format and chat-template compatibility. Task-specific model selection and distillation intuition. <i>Module B: Quantisation</i> Post-training quantisation formats: GGUF, AWQ, GPTQ, and 4/8-bit weight-only schemes. Calibration data, perplexity degradation measurement, and CPU versus GPU deployment targets. <i>Module C: Inference Engines</i> vLLM and continuous batching. PagedAttention and KV-cache management. Prefill versus decode phases, tokens-per-second under concurrency, tail latency, and autoscaling considerations. Hands-on Laboratory & Proof-of-Work (PoW) • PoW: Deploy a pre-quantised 4-bit open-weight model locally using vLLM and stress-test the endpoint to calculate tokens-per-second under increasing concurrency. • Compare quantisation formats on a fixed benchmark and report quality loss versus memory saved. • Chart the latency/throughput curve and identify the saturation point of the serving node.	30

Unit	Contents (continued)	Hrs
5	LLMOps, Evaluation & Semantic Caching <i>Module A: Observability and Tracing</i> Span-based tracing of LLM applications, prompt and response capture, token and cost attribution, latency breakdown across retrieval, generation and tool calls. <i>Module B: Evaluation Frameworks</i> Ragas metrics: faithfulness, answer relevance, context precision and recall. LLM-as-judge design, rubric construction, inter-rater agreement, and regression suites in CI. <i>Module C: Guardrails and Cost Reduction</i> Prompt injection defence, input/output filtering, PII redaction. Redis semantic caching, embedding-similarity thresholds, cache invalidation, and model routing/cascading for cost control. Hands-on Laboratory & Proof-of-Work (PoW) • PoW: Instrument a chat application with distributed tracing, route redundant queries through a semantic cache, and evaluate 50 synthetic outputs using Ragas. • Build a CI gate that fails a pull request when evaluation scores regress beyond a threshold. • Measure and report cost per resolved query before and after caching and routing.	30
Total Instructional Effort		150

4. Recommended Resources

- Official PyTorch, vLLM, Qdrant, LangGraph and Model Context Protocol documentation.
- Dao et al., *FlashAttention* and *FlashAttention-2* papers.
- Kwon et al., *Efficient Memory Management for LLM Serving with PagedAttention*.
- Lewis et al., *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*.
- Ragas documentation and the OWASP Top 10 for LLM Applications.